



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 数据结构与算法导论

实验名称: 实验 8: 排序算法的实现

专 业 班 级 通信工程 三班

学 生 学 号 _____

学 生 姓 名 _____

实 验 时 间 2025 年 5 月 25 日

实验报告

【实验目的】

掌握常用的排序方法，并掌握C语言实现排序算法的方法；了解各种排序方法过程及依据原则，并掌握各种排序方法的时间复杂度和稳定性的分析方法。

【实验原理】

本次实验，我们需要先构建学生成绩的结构体为：

```
typedef struct {  
    char name[MAX_NAME_LENGTH]; //姓名  
    int score; //分数  
    int rank; //名次  
} Student;
```

然后，我们选择快速排序这一排序方法来对学生进行排名，由于它是由二分法处理的效率很高。我们通过递归实现该算法，步骤如下：

- (1) 取基准元素
- (2) 将小于基准元素的数据移到其左边，大于基准元素的数据移到右边
- (3) 递归对两个区间进行再次排列

可以得出，该算法的复杂度为 $O(n\log n)$ 。

【实验内容与要求】

内容：

统计成绩

【问题描述】：给出n个学生的考试成绩表，每条信息由姓名和分数组成，利用排序算法完成以下任务：

- 1) 按照分数高低次序，打印出每个学生在考试中获得的名次，分数相同的为同一名次。

2) 按照名次列出每个学生的姓名和分数

【要求】学生的考试成绩需要从键盘输入数据建立，同时要设计输出格式。

要求：

- 1) 能够采用常用的排序算法中的一种实现以上两个任务；

【实验源程序】

源文件：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_NAME_LENGTH 50

typedef struct {
    char name[MAX_NAME_LENGTH];
    int score;
    int rank;
} Student;

// 快速排序
void quickSort(Student arr[], int low, int high) {
    if (low < high) {

        int pi = low;
        int pivot = arr[low].score; // 选择第一个元素作为基准值

        for (int j = low + 1; j <= high; j++) {
            // 将分数大于等于基准的元素移到左侧
            if (arr[j].score >= pivot) {
                pi++;
                Student temp = arr[pi];
                arr[pi] = arr[j];
                arr[j] = temp;
            }
        }

        // 将基准值放到正确位置
        Student temp = arr[low];
```

```

        arr[low] = arr[pi];
        arr[pi] = temp;

        quickSort(arr, low, pi - 1); // 递归排序左子数组
        quickSort(arr, pi + 1, high); // 递归排序右子数组
    }
}

int main() {
    int n;

    // 输入学生人数
    printf("请输入学生人数: ");
    scanf("%d", &n);

    // 动态分配内存存储学生信息
    Student *students = (Student *)malloc(n * sizeof(Student));
    if (students == NULL) {
        printf("内存分配失败! \n");
        return 1;
    }

    // 输入学生信息
    for (int i = 0; i < n; i++) {
        printf("请输入第 %d 个学生的姓名和分数（用空格分隔）: ", i + 1);
        scanf("%s %d", students[i].name, &students[i].score);
    }

    // 快速排序（降序）
    quickSort(students, 0, n - 1);

    // 计算名次
    if (n > 0) {
        students[0].rank = 1;
        for (int i = 1; i < n; i++) {
            if (students[i].score == students[i - 1].score) {
                students[i].rank = students[i - 1].rank; // 分数相同则名次相同
            } else {
                students[i].rank = i + 1; // 分数不同则名次为当前位置（从 1 开始）
            }
        }
    }
}

```

```
// 任务一：按分数高低输出名次
printf("\n");
printf("%-10s%-6s%-6s\n", "姓名", "分数", "名次");
printf("-----\n");
for (int i = 0; i < n; i++) {
    printf("%-10s%-6d%-6d\n", students[i].name, students[i].score,
students[i].rank);
}

// 任务二：按名次顺序列出学生
printf("\n----- 任务二结果 -----\n");
printf("%-6s%-10s%-6s\n", "名次", "姓名", "分数");
printf("-----\n");
for (int i = 0; i < n; i++) {
    printf("%-6d%-10s%-6d\n", students[i].rank, students[i].name,
students[i].score);
}

// 释放动态分配的内存
free(students);

return 0;
}
```

【实验结果】

设计输入为：

请输入学生人数：N(0<N<50)

请输入第 i 个学生的姓名和分数（用空格分隔）：姓名 成绩

输出为：

姓名	分数	名次
----	----	----

学生 1	分数 1	1
------	------	---

.....

名次	姓名	分数
----	----	----

1	学生 1	分数 1
---	------	------

.....

```
C:\Users\Eric\Desktop\数据结构实验\8\排序算法.exe
请输入学生人数: 5
请输入第 1 个学生的姓名和分数 (用空格分隔): 张三 78
请输入第 2 个学生的姓名和分数 (用空格分隔): 李四 87
请输入第 3 个学生的姓名和分数 (用空格分隔): 王五 90
请输入第 4 个学生的姓名和分数 (用空格分隔): 甲 100
请输入第 5 个学生的姓名和分数 (用空格分隔): 乙 60

姓名      分数      名次
-----
甲         100      1
王五       90       2
李四       87       3
张三       78       4
乙         60       5

----- 任务二结果 -----
名次  姓名      分数
-----
1     甲         100
2     王五       90
3     李四       87
4     张三       78
5     乙         60

-----
Process exited after 60.58 seconds with return value 0
请按任意键继续. . .
```

图 1 输出结果示例

按照我们所设想的输入与输出结构，以及我们的测试范例，我们的排序算法构建成功。

【实验心得】

本次实验，我们实现了以快速排序来对学生成绩进行统计，这里我们选用的排序算法是快速排序，这是效率最高的排序算法，但是，他带来了一些问题：

- 1. 排序算法的不稳定
快速排序是不稳定的排序算法，这意味着如果输入与输出的顺序是不会保持的，会打乱输入数据原本的排序，由于本次实验对此没有要求，我们可以使用快速排序。
- 2. 快速排序在极端情况性能的退化
当数据的大小全都一样时，快速排序的效率会大大恶化至 $O(n^2)$ 水平，所有对于大小全部相同的特例，我们可以预先判断然后退出排序。如：

```
int k = arr[low].score,sflag=0;
for (int i = low; i <= high; i++) {
    if(arr[i].score != k){
```

```

        sflag=1;
    }
}

if(sflag==0){
    return;
}

```

这样便可以优化算法。

3. 大规模数据的问题

快速排序处理大规模数据时会导致多次递归调用，可能会导致栈空间的紧张。